

数算期末 Cheat Sheet 补充

DFS序（欧拉序）与时间戳

概念

DFS序（也叫欧拉序）是对树进行DFS遍历时，记录每个节点的**进入时间** $\text{tin}[u]$ 和**离开时间** $\text{tout}[u]$ 。核心性质：

- **子树区间**：节点u的整棵子树对应DFS序中的连续区间 $[\text{tin}[u], \text{tout}[u]]$
- **祖先判定**：u是v的祖先 $\Leftrightarrow \text{tin}[u] \leq \text{tin}[v]$ 且 $\text{tout}[u] \geq \text{tout}[v]$
- **子树修改**：对u的子树整体加值 $\rightarrow$ 在DFS序数组上做区间修改 $[\text{tin}[u], \text{tout}[u]]$
- **路径查询**：配合LCA，可将树上路径问题转化为序列问题

满二叉树版（数组存储，节点编号1~n，左孩 $2i$ ，右孩 $2i+1$ ）

```
def build_dfs_order(k):
    """k层满二叉树，返回 tin 和 tout"""
    n = (1 << k) - 1
    tin = [0] * (n + 1)
    tout = [0] * (n + 1)
    timer = [0]

    def dfs(x):
        timer[0] += 1
        tin[x] = timer[0]
        if x < (1 << (k - 1)):      # 非叶节点
            dfs(2 * x)
            dfs(2 * x + 1)
        tout[x] = timer[0]

    dfs(1)
    return tin, tout
```

通用版（邻接表，任意树，迭代式防栈溢出）

```
def dfs_order(graph, root, n):
    """
    graph: 邻接表 graph[u] = [v1, v2, ...]
    返回 tin[], tout[] (1-indexed)
    """
    tin = [0] * (n + 1)
    tout = [0] * (n + 1)
    timer = 0
    vis = [False] * (n + 1)
    stack = [(root, False)]      # (节点, 是否已展开子节点)

    while stack:
        u, expanded = stack.pop()
        if expanded:
            timer += 1
            tout[u] = timer      # 离开时间
            continue
        if vis[u]:
            continue
        vis[u] = True
        timer += 1
        tin[u] = timer           # 进入时间
        stack.append((u, True))  # 回溯时记录离开时间
        for v in reversed(graph[u]):
            if not vis[v]:
                stack.append((v, False))

    return tin, tout
```

递归版（更简洁，小规模适用）

```

import sys
sys.setrecursionlimit(200000)

tin = [0] * (n + 1)
tout = [0] * (n + 1)
timer = 0

def dfs(u, parent):
    global timer
    timer += 1
    tin[u] = timer
    for v in graph[u]:
        if v != parent:
            dfs(v, u)
    timer += 1
    tout[u] = timer

dfs(root, -1)

```

带parent记录的版本（常配合LCA使用）

```

def dfs_order_with_parent(graph, root, n):
    tin = [0] * (n + 1)
    tout = [0] * (n + 1)
    parent = [0] * (n + 1)
    depth = [0] * (n + 1)
    timer = 0
    stack = [(root, 0, 0)]  # (节点, 父, 深度)

    while stack:
        u, fa, d = stack.pop()
        if tin[u]:  # 已访问 → 记离开时间
            timer += 1
            tout[u] = timer
            continue
        timer += 1
        tin[u] = timer
        parent[u] = fa
        depth[u] = d
        stack.append((u, fa, d))  # 再次入栈记离开
        for v in reversed(graph[u]):
            if v != fa:
                stack.append((v, u, d + 1))

    return tin, tout, parent, depth

```

典型应用

应用1：判断祖先关系 $O(1)$

```

def is_ancestor(u, v):
    return tin[u] ≤ tin[v] and tout[u] ≥ tout[v]

```

应用2：子树求和（DFS序 + 树状数组）

将树拍平为DFS序后，"对u子树所有节点加val" = 对序列 `[tin[u], tout[u]]` 区间加val。

```
# 建树 + DFS序
tin, tout = dfs_order(graph, root, n)

# 树状数组维护DFS序数组
tree = [0] * (2*n + 2)
def update(idx, val):
    while idx <= 2*n:
        tree[idx] += val; idx += idx & -idx
def query(idx):
    s = 0
    while idx > 0:
        s += tree[idx]; idx -= idx & -idx
    return s

# 子树u整体加val
def subtree_add(u, val):
    update(tin[u], val)
    update(tout[u] + 1, -val)

# 查询节点v的当前值
def point_query(v):
    return query(tin[v])
```

应用3: 子树内节点计数/查询

DFS序把子树映射为连续区间，可以用任何区间数据结构（前缀和、线段树、BIT）处理子树问题。

应用4: 配合LCA处理路径

树上路径 $u \rightarrow v$ 可分解为 $u \rightarrow \text{LCA}$ 和 $v \rightarrow \text{LCA}$ 两段，每段在DFS序上用差分处理。

DFS序 vs 前/中/后序 对比

遍历方式	记录时机	用途
前序	进入时记录	建树（前序+中序→唯一树）
后序	离开时记录	拓扑排序、SCC
DFS序（欧拉序）	进入+离开都记录	子树区间操作、祖先判定

关键区别：前序/后序只记一次（n个时间戳），DFS序记两次（2n个时间戳），额外的信息让你能确定子树边界。

算法模板补充：高频缺失知识点

1. 双指针 / 滑动窗口通用模板

双指针（Two Pointers）与滑动窗口（Sliding Window）常用于数组/字符串的子数组或子串问题，能够在 $O(n)$ 时间内解决。

1.1 双指针（相向 / 同向）

相向双指针：一般用于排序数组的两数之和、回文判断等。

```
def two_sum_sorted(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:
        s = nums[left] + nums[right]
        if s == target:
            return [left, right]
        elif s < target:
            left += 1
        else:
            right -= 1
    return []
```

同向双指针（快慢指针）：用于链表环检测、移除重复元素等。

```
def remove_duplicates(nums):
    if not nums:
        return 0
    slow = 0
    for fast in range(1, len(nums)):
        if nums[fast] != nums[slow]:
            slow += 1
            nums[slow] = nums[fast]
    return slow + 1
```

1.2 滑动窗口（固定长度 / 可变长度）

固定长度窗口：窗口长度固定为 k ，维护窗口内信息（和、最大值等）。

```
def fixed_window_sum(nums, k):
    # 长度为 k 的所有子数组的最大和
    window_sum = sum(nums[:k])
    max_sum = window_sum
    for i in range(k, len(nums)):
        window_sum += nums[i] - nums[i - k]
        max_sum = max(max_sum, window_sum)
    return max_sum
```

可变长度窗口（求最小覆盖子串等）：右指针扩张，左指针收缩以维护条件。

```
def min_window(s, t):
    # 返回 s 中包含 t 所有字符的最短子串
    from collections import Counter, defaultdict
    need = Counter(t)
    window = defaultdict(int)
    left = right = 0
    valid = 0
    start, length = 0, float('inf')

    while right < len(s):
        c = s[right]
        right += 1
        if c in need:
            window[c] += 1
            if window[c] == need[c]:
                valid += 1
        # 收缩窗口
        while valid == len(need):
            if right - left < length:
                start = left
                length = right - left
            d = s[left]
            left += 1
            if d in need:
                if window[d] == need[d]:
                    valid -= 1
                window[d] -= 1
    return s[start:start+length] if length != float('inf') else ""
```

2. 折半搜索（Meet in the Middle）

当问题规模较大（如 $n \leq 40$ ）但直接枚举 2^n 不可行时，可将集合分为两半，分别枚举后合并。

典型应用：子集和等于目标值的方案数、最大子集和不超过限制等。

```
def meet_in_the_middle(nums, target):
    # 求有多少子集的和等于 target (nums 长度最大 40)
    n = len(nums)
    left = nums[:n//2]
    right = nums[n//2:]

    # 枚举一半的所有子集和
    def get_sums(arr):
        sums = []
        m = len(arr)
        for mask in range(1 << m):
            s = 0
            for i in range(m):
                if mask >> i & 1:
                    s += arr[i]
            sums.append(s)
        return sums

    left_sums = get_sums(left)
    right_sums = get_sums(right)
    right_sums.sort()

    count = 0
    for ls in left_sums:
        need = target - ls
        # 二分查找 need 在 right_sums 中的出现次数
        lo = bisect_left(right_sums, need)
        hi = bisect_right(right_sums, need)
        count += (hi - lo)
    return count
```

优化：使用 bisect 模块加速；若需要输出具体方案，可额外记录掩码组合。

3. 状态压缩 BFS

普通 BFS 处理状态时，若状态可以用位掩码表示（如某些格子是否走过、开关状态等），可使用状态压缩 BFS。常用于最短路径 + 状态约束（如钥匙和门、网格变形）。

经典问题：迷宫中有钥匙（a~f）和门（A~F），求从起点到终点的最少步数。

```

from collections import deque

def shortest_path_with_keys(grid):
    """
    grid: List[str], 包含 'S' 起点, 'E' 终点, '.' 空地, '#' 墙,
        'a'-'f' 钥匙, 'A'-'F' 门
    返回最少步数, 无法到达返回 -1
    """
    R, C = len(grid), len(grid[0])
    # 找到起点
    sr = sc = -1
    for i in range(R):
        for j in range(C):
            if grid[i][j] == 'S':
                sr, sc = i, j
                break
    if sr != -1:
        break

    # 状态 (r, c, keys) keys 为位掩码, 第0位表示钥匙 'a'
    start = (sr, sc, 0)
    dist = {start: 0}
    q = deque([start])
    dirs = [(1,0), (-1,0), (0,1), (0,-1)]

    while q:
        r, c, keys = q.popleft()
        if grid[r][c] == 'E':
            return dist[(r, c, keys)]
        for dr, dc in dirs:
            nr, nc = r + dr, c + dc
            if not (0 ≤ nr < R and 0 ≤ nc < C):
                continue
            cell = grid[nr][nc]
            if cell == '#':
                continue
            # 遇到门: 检查是否有对应钥匙
            if 'A' ≤ cell ≤ 'F':
                key_needed = 1 << (ord(cell) - ord('A'))
                if not (keys & key_needed):
                    continue
            new_keys = keys
            # 捡钥匙
            if 'a' ≤ cell ≤ 'f':
                new_keys |= (1 << (ord(cell) - ord('a')))
            state = (nr, nc, new_keys)
            if state not in dist:
                dist[state] = dist[(r, c, keys)] + 1
                q.append(state)
    return -1

```

注意： 钥匙种类不超过 6 时 ($2^6=64$ 种状态)，内存和时间均可接受。

4. 数论基础（质因数分解、约数枚举、筛法）

4.1 质因数分解（试除法）

```

def prime_factors(n):
    """返回 n 的质因数分解字典 {质因子: 指数}"""
    factors = {}
    i = 2
    while i * i ≤ n:
        while n % i == 0:
            factors[i] = factors.get(i, 0) + 1
            n //= i
        i += 1 if i == 2 else 2 # 可优化跳过偶数
    if n > 1:
        factors[n] = factors.get(n, 0) + 1
    return factors

```

4.2 枚举所有约数

```
def get_divisors(n):
    """返回 n 的所有正约数（未排序）"""
    divisors = []
    i = 1
    while i * i ≤ n:
        if n % i == 0:
            divisors.append(i)
            if i ≠ n // i:
                divisors.append(n // i)
        i += 1
    return divisors # 可排序: sorted(divisors)
```

4.3 筛法（埃氏筛 / 欧拉筛）

埃拉托斯特尼筛法（求 $[1, n]$ 内所有素数）：

```
def sieve_eratosthenes(n):
    is_prime = [True] * (n + 1)
    is_prime[0] = is_prime[1] = False
    for i in range(2, int(n**0.5) + 1):
        if is_prime[i]:
            for j in range(i*i, n+1, i):
                is_prime[j] = False
    primes = [i for i, val in enumerate(is_prime) if val]
    return primes
```

欧拉筛（线性筛）：每个合数只被最小质因子筛一次，复杂度 $O(n)$ 。

```
def sieve_linear(n):
    primes = []
    is_prime = [True] * (n + 1)
    for i in range(2, n + 1):
        if is_prime[i]:
            primes.append(i)
            for p in primes:
                if i * p > n:
                    break
                is_prime[i * p] = False
                if i % p == 0:
                    break
    return primes
```

5. 懒删除堆（Lazy Deletion Heap）

需要支持删除任意元素（非堆顶）的堆结构，通常用优先队列配合延迟删除字典实现。

应用场景：动态求中位数、滑动窗口最大/最小值（但后者更推荐单调队列）、多路归并中的元素淘汰等。

```

import heapq

class LazyHeap:
    def __init__(self):
        self.heap = []
        self.deleted = {} # 记录待删除元素及其数量
        self.size = 0

    def push(self, val):
        heapq.heappush(self.heap, val)
        self.size += 1

    def _lazy_remove(self):
        """清理堆顶已被标记删除的元素"""
        while self.heap and self.heap[0] in self.deleted:
            top = self.heap[0]
            self.deleted[top] -= 1
            if self.deleted[top] == 0:
                del self.deleted[top]
            heapq.heappop(self.heap)

    def pop(self):
        self._lazy_remove()
        if self.heap:
            self.size -= 1
            return heapq.heappop(self.heap)
        raise IndexError("pop from empty heap")

    def remove(self, val):
        """标记删除某个值（不要求 val 一定在堆中）"""
        self.deleted[val] = self.deleted.get(val, 0) + 1
        self.size -= 1

    def top(self):
        self._lazy_remove()
        return self.heap[0] if self.heap else None

    def __len__(self):
        return self.size

```

使用示例（动态维护窗口内的元素，支持移除任意元素）：

```

heap = LazyHeap()
heap.push(5)
heap.push(3)
heap.push(8)
heap.remove(3)
print(heap.pop()) # 5
heap.push(2)
print(heap.pop()) # 2

```

6. 树上背包 DP（树形依赖背包）

在树上做分组背包，通常用于「选择若干节点，满足父子依赖关系，使得总价值最大/最小」的问题。常见于「有依赖的背包问题」和「树形 DP + 体积限制」。

经典问题：给定一棵树，每个节点有重量 $w[i]$ 和价值 $v[i]$ ，若选择某个节点，则必须选择其父节点。在总重量不超过容量的情况下，求最大价值。


```
def tree_knapsack(edges, w, v, capacity):
    """
    edges: List[(parent, child)], 默认 0 为根节点
    w: List[int] 每个节点的重量（体积）
    v: List[int] 每个节点的价值
    capacity: 背包容量
    返回最大价值（根节点必选时）
    """
    n = len(w)
    g = [[] for _ in range(n)]
    for par, ch in edges:
        g[par].append(ch)

    # 返回 dp 数组, dp[j] 表示以当前节点为根的子树内, 体积为 j 时能获得的最大价值
    def dfs(u):
        # 初始化: 必须选 u 自己
        dp = [0] * (capacity + 1)
        for j in range(w[u], capacity + 1):
            dp[j] = v[u] # 自己必须选

        for child in g[u]:
            child_dp = dfs(child) # 子树的dp
            # 分组背包: 遍历当前已合并的容量, 再遍历子树的容量 (倒序防止重复)
            for j in range(capacity, w[u] - 1, -1):
                for k in range(capacity - j + 1):
                    if j + k ≤ capacity:
                        dp[j + k] = max(dp[j + k], dp[j] + child_dp[k])

        return dp

    root_dp = dfs(0)
    return max(root_dp) # 返回不超过容量的最大价值
```

注意: 如果根节点不必须选, 可以引入一个虚拟根节点 0, 重量和价值均为 0, 且与所有原本的根相连。

纯 0-1 背包 (非树上) 也作为补充:

```
def zero_one_knapsack(w, v, capacity):
    n = len(w)
    dp = [0] * (capacity + 1)
    for i in range(n):
        for j in range(capacity, w[i] - 1, -1):
            dp[j] = max(dp[j], dp[j - w[i]] + v[i])
    return dp[capacity]
```